

A MIPS32 Soft Processor on Cyclone V

FPGA Implementation, HPS Monitoring, and Hardware Demonstration

Hanan Quispe

May 2026

Introduction

Objective

- Implement a *MIPS32-style soft processor* in VHDL on the *DE10-Standard Cyclone V SoC*.
- Support a useful ISA subset: *add, sub, and, or, slt, addi, lw, sw, beq*.
- Build a *complete demonstrator*, not only a datapath:
 - instruction loading from Linux on the HPS,
 - runtime observation on the LCD and seven-segment displays,
 - end-to-end validation with assembly programs.
- Main result: a working *3-stage staged processor* adapted to FPGA synchronous memories.

Outline

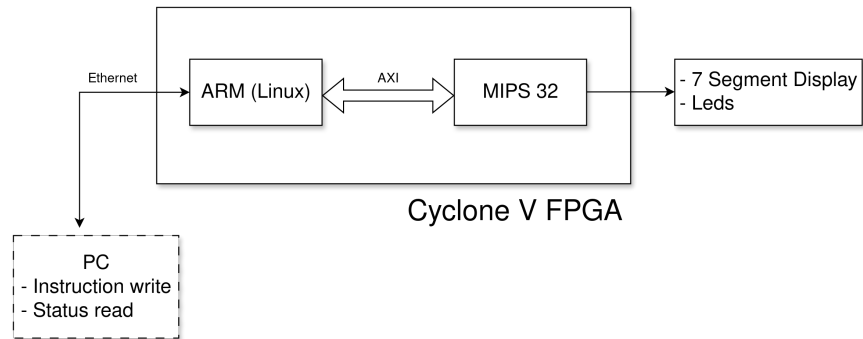
1. Context and design choices
2. Implemented architecture
3. Development and integration
4. Validation and live demo
5. Theoretical discussion
6. Conclusion and future work

Context

Why This Project?

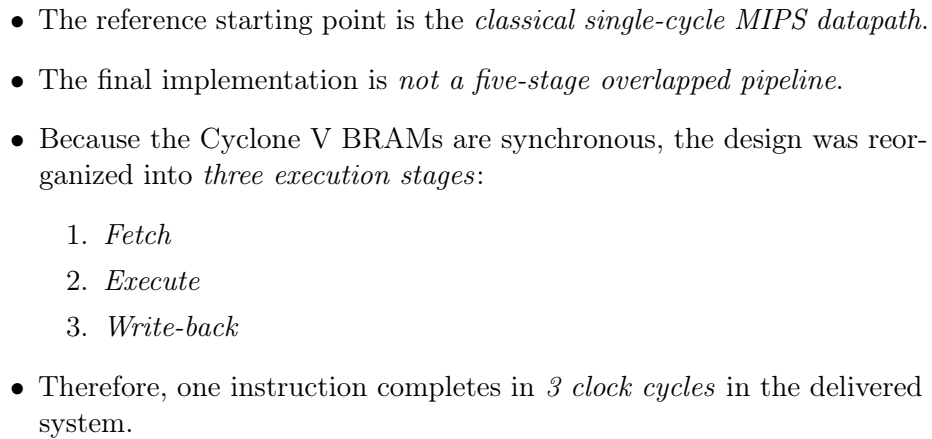
- MIPS remains a *clear teaching ISA* for understanding datapaths, control, and memory access.
- The project connects *computer architecture* with *real FPGA implementation constraints*.
- The Cyclone V SoC makes it possible to combine:
 - a processor implemented in FPGA fabric,
 - an ARM HPS used for loading, debugging, and monitoring.
- The goal was to go beyond simulation and obtain a *board-level demonstrator*.

System-Level View



- FPGA side: MIPS core, instruction memory, data memory, observability logic.
- HPS side: program loading, state readback, LCD monitor.
- Result: a complete HW/SW co-design around the processor.

Target vs Implemented Organization



- *32-bit datapath* with a *32 x 32-bit register file*.
- *Harvard organization*: separate instruction and data memories.
- Main blocks:
 - program counter,

- register file,
- main control and ALU control,
- ALU,
- sign extension and address-generation logic,
- BRAM-backed instruction and data memories.
- Supported instruction subset:
 - R-type: *add, sub, and, or, slt*
 - I-type: *addi, lw, sw, beq*

Why a 3-Stage Execution Model?

- In the textbook single-cycle view, fetch, decode, execute, memory access, and architectural update all fit in one cycle.
- On this FPGA, *instruction and data memories are synchronous BRAMs*.
- This means memory outputs are not immediately available in the same combinational path.
- The practical solution was to serialize execution into:
 - *Fetch*: read instruction memory,
 - *Execute*: decode, read registers, compute ALU result,
 - *Write-back*: update register file or finalize memory effect.
- This keeps the architecture simple and robust for demonstration.

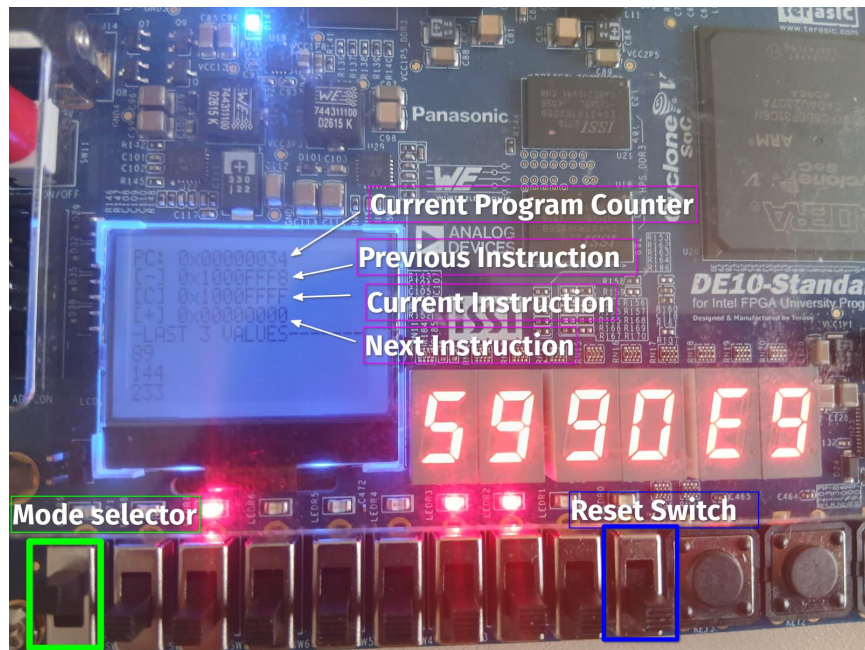
Platform Integration

HPS-FPGA Interface and Observability

- The HPS accesses the processor through the *lightweight HPS-to-FPGA AXI bridge*.
- This enables Linux user-space tools to:
 - load instruction memory,
 - inspect data memory,
 - read exported status registers.

- Board peripherals are used as debug interfaces:
 - *LCD*: current PC and nearby instructions,
 - *seven-segment displays*: recent stored values,
 - *LEDs*: execution state and control information.

Demonstrator on the Board



- *Manual mode*: step through fetch, execute, and write-back.
- *Automatic mode*: run continuously.
- The *switcher* block controls when state updates are allowed.
- A small FIFO keeps the *three latest stored values* for visual observation.

Development

Development Methodology

- The project followed a *bottom-up approach*:
 1. validate elementary blocks in simulation,

2. integrate the datapath,
 3. connect memories and HPS interfaces,
 4. validate the complete system on hardware.
- Main tools:
 - *VHDL* for RTL,
 - *Quartus* and *Platform Designer* for synthesis and SoC integration,
 - *ModelSim* for unit simulation,
 - *Python* for the assembler,
 - *C* for HPS-side utilities.

Main Building Blocks Delivered

- *Program counter* with reset and enable.
- *Register file* with 2 read ports, 1 write port, and hardwired register 0.
- *Main control* for opcode decoding.
- *ALU control* and *32-bit ALU*.
- *Instruction and data BRAMs*.
- *Switcher* for three-stage execution control.
- *FIFO and status exports* for observability from the HPS and the board peripherals.

Validation

Validation Strategy

- *Unit-level validation* in simulation:
 - ALU,
 - ALU control,
 - register file,
 - program counter,
 - main control.

- *Integration validation:*
 - HPS-to-FPGA AXI bridge read/write tests,
 - instruction-memory programming from Linux,
 - data-memory readback.
- *System-level validation:*
 - Fibonacci program,
 - mixed ISA validation program,
 - LCD and seven-segment observation on the board.

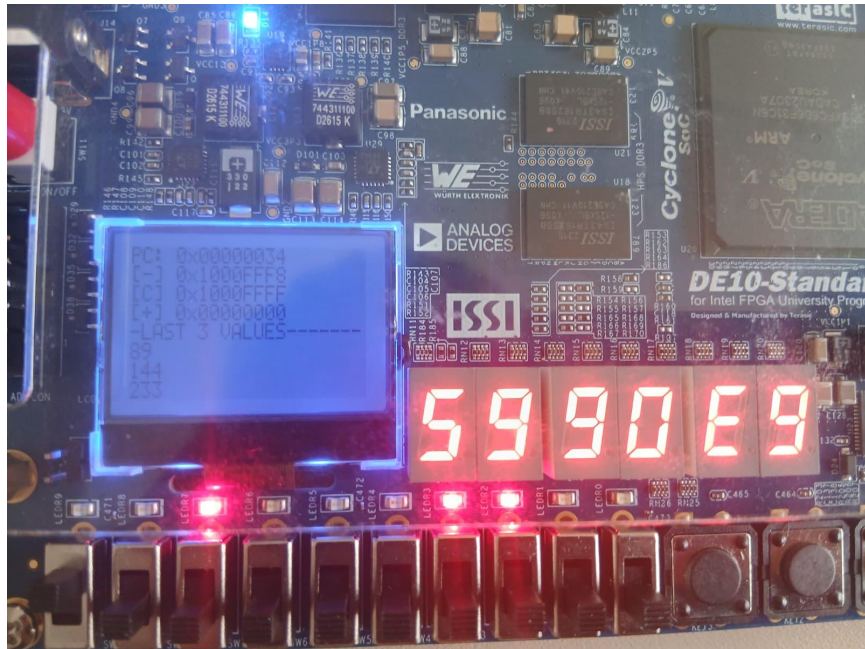
Demo Scenario

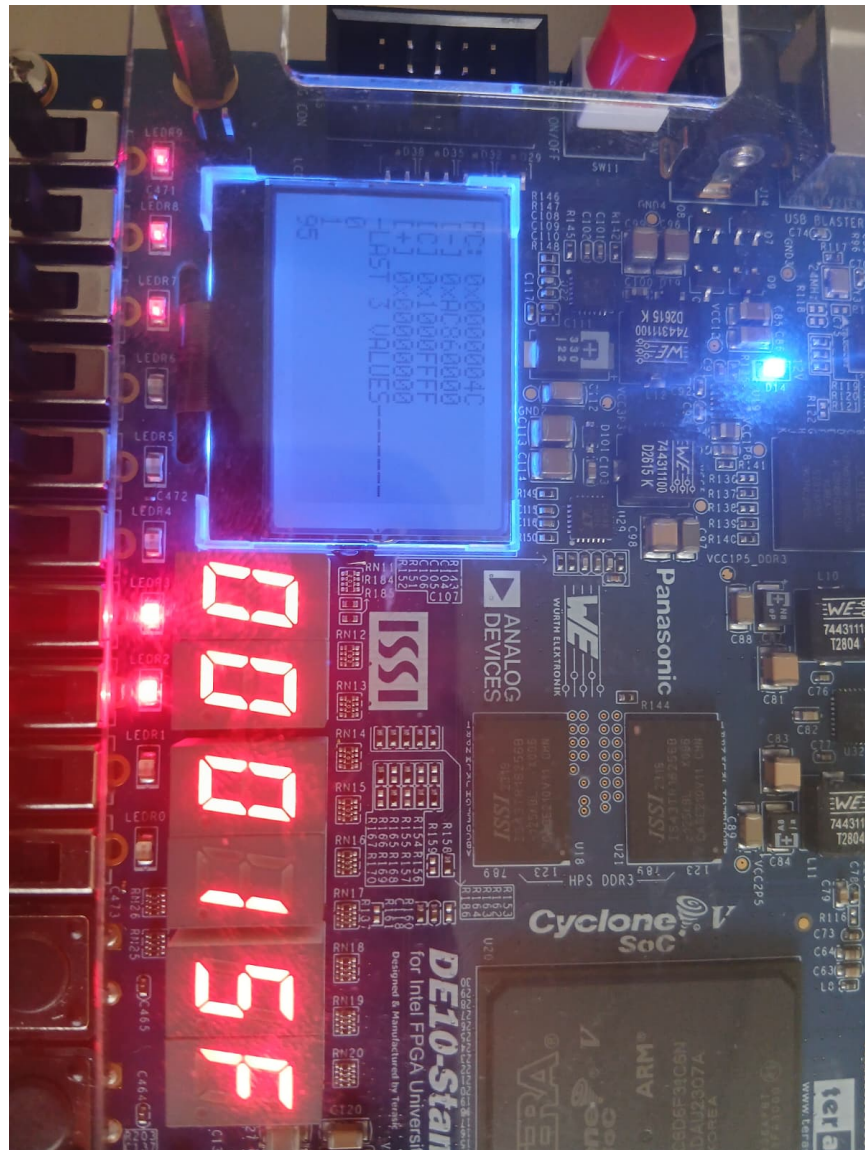
- Assemble the program with the custom Python assembler.
- Load the generated machine code into *instruction memory* from the HPS.
- Run the processor on the FPGA in *manual* or *automatic* mode.
- Observe:
 - *program flow* on the LCD,
 - *recent memory stores* on the seven-segment displays,
 - *memory contents* through HPS-side utilities.
- This validates the entire chain from assembly source to hardware execution.

Fibonacci Demonstration

- The main demonstrator stores successive Fibonacci values into data memory.
- Why this program is useful:
 - repeated arithmetic operations,
 - loop control with *beq*,
 - repeated *sw* operations,
 - easy numerical verification through memory readback.
- Expected sequence begins with:
 - 0, 1, 1, 2, 3, 5, 8, ...

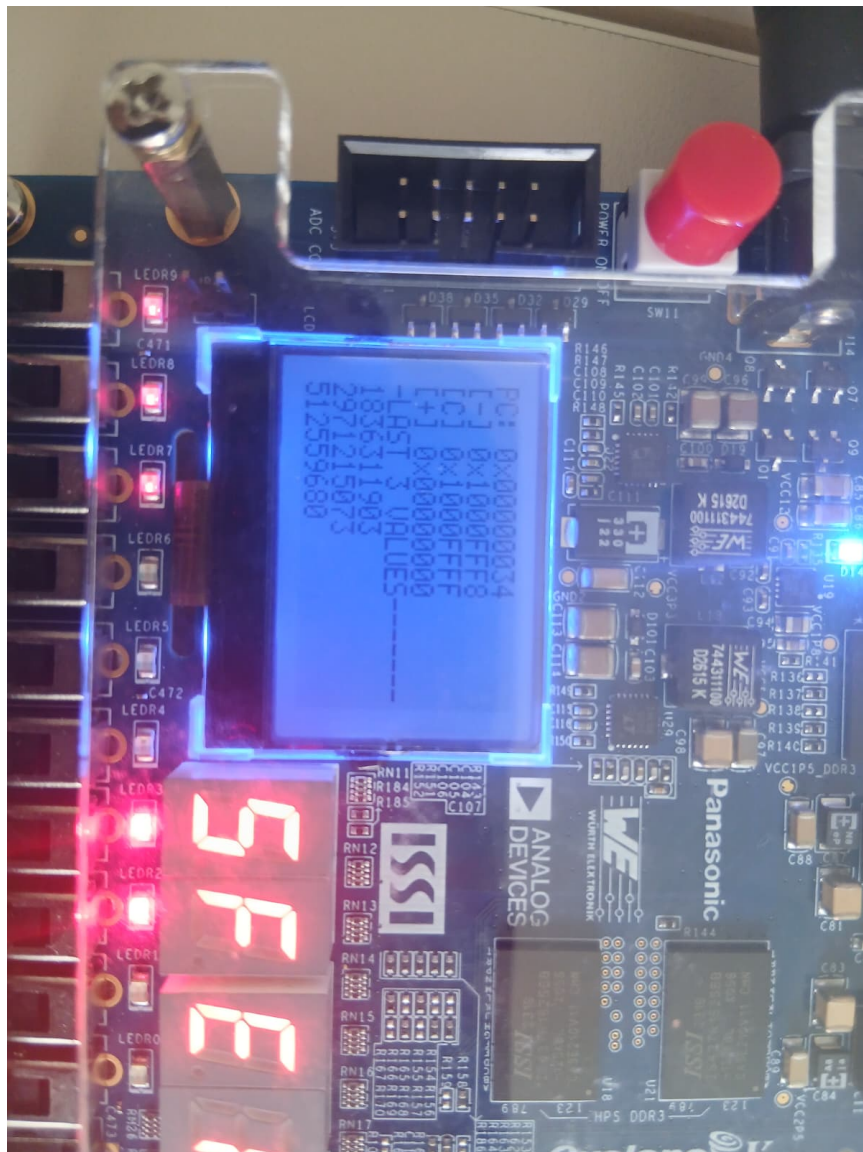
Demo Evidence





- Left: Fibonacci execution on the board.
- Right: separate ISA validation program.
- These images show that the demonstrator is not limited to one benchmark only.

Observed Limitation



- The datapath and register file are 32 bits wide.
- Fibonacci eventually exceeds 2^{32} .
- The stored result then *wraps around modulo* 2^{32} .

- This is not a bug in the demonstrator: it is the expected consequence of a 32-bit implementation.

Resource Utilization

- Post-fit utilization on *Cyclone V 5CSXFC6D6F31C6*:
 - 2,000 ALMs out of 41,910 available, about 5%,
 - 3,230 registers,
 - 65,536 block memory bits,
 - 8 RAM blocks,
 - 0 DSP blocks.
- Conclusion: the processor and its observability infrastructure fit comfortably on the target FPGA.

Theoretical Discussion

Why Harvard for a 1-CPI Objective?

- In a 5-stage pipelined MIPS, instruction fetch and data memory access may be needed in the same cycle.
- With *one shared memory*, this creates a *structural hazard*.
- A *Harvard architecture* removes this conflict by separating instruction and data memories.
- Even though the delivered core is not yet fully pipelined, this design choice is already present and remains the correct one for a future *1-CPI* target.

RAW Hazard After *lw*

- Example:

```

1    lw    $t0, 0($t1)
2    add   $t2, $t0, $t3
```

- In a classical pipeline, the loaded data becomes available too late for the immediately following instruction.
- Standard solutions:

- *forwarding* when possible,
 - *stalling* for the specific *lw-use* case.
- In the delivered processor, execution is *serialized*, so this hazard does not appear in the same way.

Branch Delay Slot and Control in VHDL

- *Branch delay slot*:
 - historical MIPS feature,
 - important for the originally planned pipelined version,
 - not exposed as a visible architectural contract in the delivered demonstrator.
- *Control generation in one cycle*:
 - the control unit is *combinational logic*,
 - opcode and function bits propagate through logic equations,
 - signals settle during the clock period before sequential blocks sample them.

Conclusion

Conclusion

- The project produced a *working MIPS32-style processor demonstrator* on Cyclone V.
- The final contribution is both:
 - a functional processor core,
 - a complete experimentation environment with assembler, flasher, HPS tools, LCD monitor, and board-level observability.
- The implementation is *functionally operational* and *hardware-demonstrated*.
- The key architectural adaptation was moving from the textbook single-cycle view to a *practical 3-stage execution model* compatible with FPGA memories.

Future Work

- Move from staged execution to a *true pipelined architecture*.
- Add *hazard detection and forwarding*.
- Extend the ISA, especially jumps and additional immediate operations.
- Improve performance analysis and timing closure.
- Strengthen the HPS monitor with richer debug information.

Thank You

- Questions?
- Contact: hanan.quispe-condori@polytechnique.edu